ISE-A2 | Domain Model and Testable Specification

Assignment Deadline Change Tracking and Verification System

Course: Intelligent Software Engineering | Assignment released: 5 October 2026 | Due: 12 October 2026, 16:30 (Asia/Shanghai)

1. Objective and system boundary

A1 investigated the risk that a student's manually tracked coursework deadline may not reflect a later official course notice. A2 formalizes that narrow workflow. The system stores a student-owned deadline, links proposed changes to attributable evidence, and requires an authorized review before creating a new tracked version. It does not alter the university's official deadline, access a live LMS, or infer authenticity from arbitrary document text.

Specification question: Under which domain states and evidence conditions can a proposed deadline replace the current personal record without losing history or bypassing the student?

A1 scenario	Domain behavior in A2	Verification
N1 Clear extension	Verified source → pending candidate → owner confirms → versioned update	I1, I2, I3; successful walkthrough
B1 Conflicting dates	Two sources / stale base version → reject unsafe update; preserve state	I3; VERSION_CONFLICT
A1 Malicious claim	Untrusted source may be recorded, but cannot authorize an update	I2; SOURCE_NOT_AUTHORIZED

2. Domain language, boundaries and events

Key entities are Assignment (aggregate root), EvidenceSource, ChangeCandidate, HistoryEntry and the student owner. DeadlineValue is an immutable value object containing an ISO 8601 timestamp with UTC offset and an IANA time-zone name. The full glossary and field-level dictionary appear in Domain-Data-Dictionary-and-Rules.xlsx.

Category	Members / meaning
Entities	Student, Course, Assignment, EvidenceSource, ChangeCandidate; HistoryEntry is an immutable domain record.
Value objects	DeadlineValue (local timestamp and named time zone); ReviewDecision (CONFIRM or REJECT).
Domain events	CandidatePendingReview, CandidateInvalid, CandidateRejected, DeadlineChangeApplied are emitted by the reference model. Assignment registration and source receipt are conceptual input events.
Trust boundaries	Instructor/LMS establishes academic authority; tracker stores attributed claims; owner decides personal update; calendar is external.

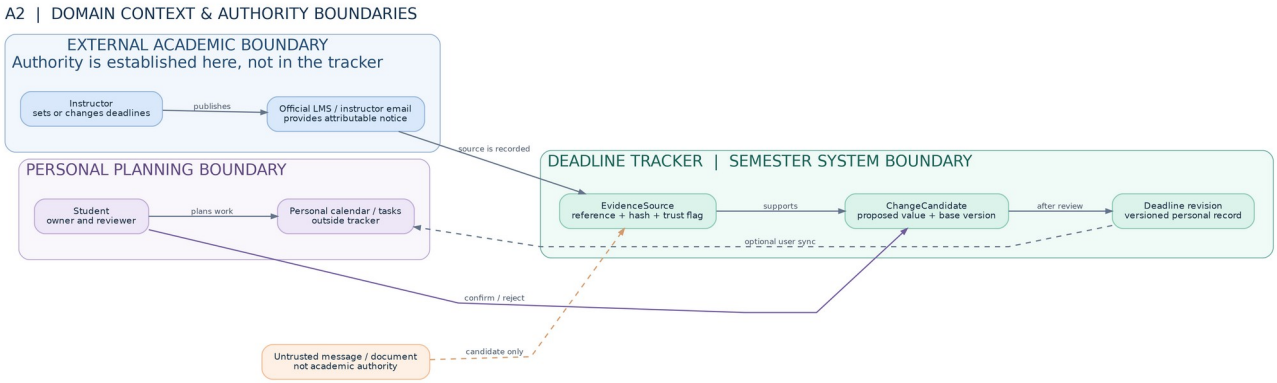


Figure 1. Bounded context and authority map. Full-resolution vector graphic is included in diagrams/context.svg.

3. Process model and state transitions

The process separates detection from acceptance. A well-formed candidate with a different timestamp becomes PENDING_REVIEW. An invalid candidate cannot modify the deadline. An owner rejection terminates the candidate. Owner confirmation succeeds only when the linked source is verified official and the candidate base version still matches the assignment.

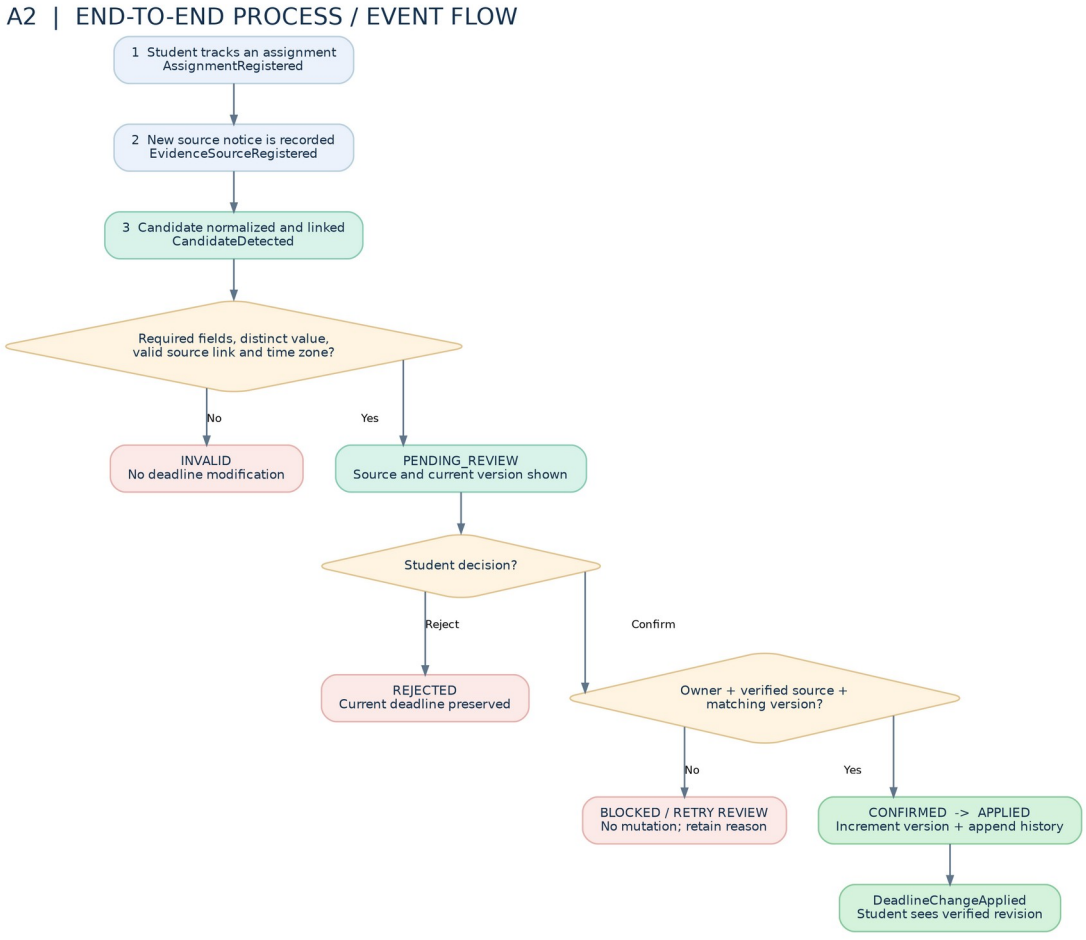


Figure 2. Event flow including validation failures, rejection, blocked confirmation, and successful versioning.

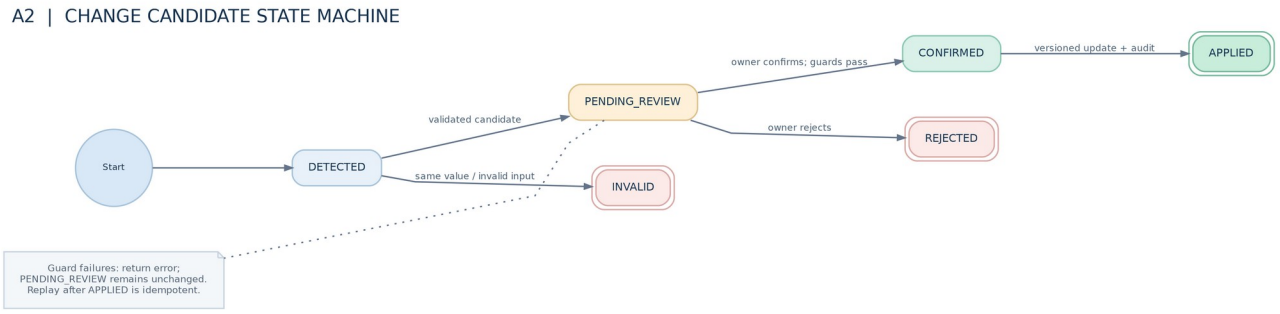


Figure 3. Candidate lifecycle. A failed guard returns an error without advancing PENDING_REVIEW.

4. Data dictionary and consistency

The workbook defines 31 fields across Assignment, DeadlineValue, EvidenceSource, ChangeCandidate, HistoryEntry and DomainEvent, including type, required status, constraints, origin, privacy classification and proposed retention. The model stores local offset-aware timestamps and compares their UTC instants. Assignment.version starts at 1 and increments only for a newly applied revision.

Key relationship	Constraint
Assignment → Student	Exactly one owner for each student-maintained assignment record.
ChangeCandidate → Assignment + EvidenceSource	Both references must exist and source.assignment_id must match candidate.assignment_id.
HistoryEntry → applied candidate	A successful confirmation creates one immutable entry with old_version + 1 = new_version.
EvidenceSource → authority	source kind and externally verified flag are checked; hash alone does not authenticate origin.

5. Invariants with violating examples

ID / rule	Violation example	Independent automated oracle
I1 Confirmation and owner	A non-owner confirms a detected candidate.	ACTOR_NOT_OWNER; deadline and history remain unchanged.
I2 Provenance and trust	A chat message or missing source is treated as official.	SOURCE_NOT_AUTHORIZED / INVALID_LINK; no update.
I3 Version and history	A stale candidate is applied or one update appends two revisions.	VERSION_CONFLICT, version +1, one history entry, idempotent replay.
I4 No silent mutation	A rejected candidate nevertheless changes the calendar deadline.	Before/after deadline, version, and history equality.
I5 Temporal validity	A naive local date or mismatched time-zone offset is accepted.	NAIVE_DATETIME or OFFSET_MISMATCH error.

Each invariant is mapped to exact tests in the Invariants worksheet. The tests do not call methods that simply return a claimed compliance flag; they inspect state and errors independently.

6. Material rule: decision table and assertion

Material rule M1: an existing tracked deadline is updated only after the owning student confirms a pending candidate whose linked source has externally verified official provenance and whose base version matches the current assignment. Rejection never mutates the deadline.

Rule	Decision / precondition	Expected action
R1	Confirm; owner; PENDING; verified official; current base version	Apply exactly once; increment and record history
R2	Reject; owner; PENDING	Mark REJECTED; keep current record
R3	Confirm by non-owner	ACTOR_NOT_OWNER; no mutation
R4	Confirm with untrusted/unverified source	SOURCE_NOT_AUTHORIZED; no mutation
R5	Confirm with stale base version	VERSION_CONFLICT; no mutation
R6–R9	Illegal state, idempotent retry, missing links, same value	Block, replay safely or mark INVALID as specified

Formal precondition for application: candidate.status = PENDING_REVIEW ∧ reviewer = assignment.student_id ∧ source.authority_verified ∧ source.kind ∈ {OFFICIAL_LMS, INSTRUCTOR_EMAIL} ∧ candidate.base_version = assignment.version ∧ source.assignment_id = assignment.id ∧ proposed UTC instant ≠ current UTC instant. Postcondition: assignment.version′ = version + 1, one HistoryEntry is appended, candidate.status′ = APPLIED.

7. Data governance: source, authority, quality, retention and privacy

Boundary	Explicit specification / limitation
Source / authority	Only student-supplied synthetic fixtures are used. Real LMS/email notices remain institutionally authoritative outside the tracker; the test fixture authority_verified flag is an assumption, not authenticated evidence.
Quality	Require required identifiers, existing links, SHA-256 integrity format, offset-aware timestamp, matching IANA offset, distinct proposed value and optimistic version check.
Privacy / access	Student is the logical owner; no raw PDFs, emails, chat messages or identifying student information are persisted in the reference model. Public repositories contain only synthetic identifiers.
Retention proposal	If deployed: source/candidate metadata until course end +30 days; revision history until course end +90 days; erase personal workspace on authorized deletion. These are design defaults subject to institutional/legal approval.
Deletion boundary	The reference store is in-memory and has no deployed deletion workflow or backups. Production removal must propagate through indexes, logs and backups; deletion approval requires privacy review.

8. Model walkthrough and verification record

Reproduction environment: Python 3.11 or later, standard library only, fixed synthetic fixture dataset a2-synthetic-fixtures-v1, no randomness and no network calls. Run from repository root: python -m unittest discover -s tests -v. Actual output is preserved in evidence/test-results.txt.

Path	Trigger	Observed outcome
W1 Normal	Pending candidate; owner confirms official verified notice	State APPLIED, version 2, one old→new history entry.
W2 Boundary	Two proposals from version 1; second applied after first	VERSION_CONFLICT; first revision remains current.
W3 Abuse	Unverified chat source requests confirmation	SOURCE_NOT_AUTHORIZED; no update.
W4 Rejection	Owner rejects a pending candidate	REJECTED; current deadline and history unchanged.
W5 Known limitation	Direct mutation of public store.assignments dictionary	Expected failing test documents missing encapsulation.

Verified run: 18 tests executed, 17 passed, one marked expected failure. The expected failure is not an unanticipated testing error: direct programmatic mutation of an exposed in-memory dictionary bypasses domain methods. The model consequently demonstrates domain rules under its guarded API but not production authorization, persistence transactions, identity authentication, source authenticity, or guaranteed deletion.

Traceability and reproduction checks

Test evidence is retained verbatim rather than summarized as an unverified claim. Tests use the Python standard library and a frozen JSON fixture containing original, proposed and conflicting deadlines at 16:30 in Asia/Shanghai. The scenario does not access external websites or personal student records.

Evidence	Location / command
Frozen synthetic input	data/synthetic-fixtures.json; dataset_id = a2-synthetic-fixtures-v1
Test oracle / violating examples	tests/test_domain.py; checks use assertRaises and inspect deadline/version/history
Recorded execution	evidence/test-results.txt; run python -m unittest discover -s tests -v
Formal rules	Domain-Data-Dictionary-and-Rules.xlsx, Rules and Invariants worksheets

9. Conclusion

A2 converts the A1 deadline-change workflow into a bounded, inspectable domain model. The specification provides explicit language, diagrams, 31 data fields, five invariants, a nine-row decision table, and independent executable checks. The central guarantee is that untrusted, unreconciled or unapproved deadline changes do not alter the tracked record through the guarded domain workflow. Deployment would additionally require access-controlled persistence, authentic source verification, institutional privacy approval and measured integration behavior.